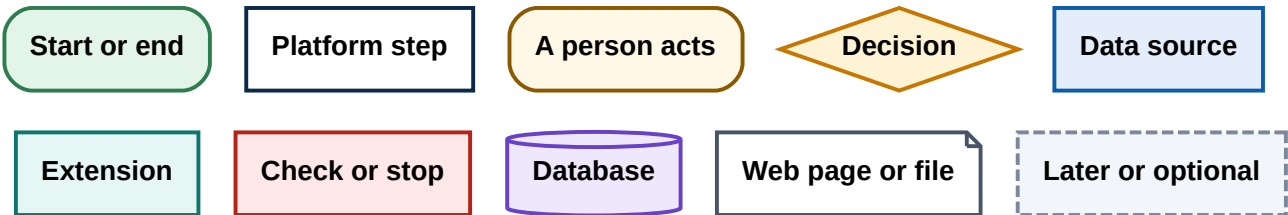


seo-advisor: requirements and architecture

This document gives the requirements, the architecture and the start plan for **seo-advisor**, a platform that audits, improves and manages the SEO content of several websites. It starts with **one page**: a person gives the URL of one page and a keyword, and the tool does SEO on that page. The whole site comes later. Two example sites, MyPipit and extendmy.life, are used to test it. The tool is general and is not shaped around these two sites. Read this document before you write code. The short work plan for Claude Code is `PLAN.md` at the repo root, next to `CLAUDE.md`. This PDF holds the same plan with reasons and diagrams.

Item	Value
Date	5 October 2026. Revised 8 October 2026 (single page first).
Status	Draft for review. Built: the safe fetcher, the MyPipit sitemap list, the tables tenants and sites, the API and the web app. Next: Phase 1, slice 2 (fetch one MyPipit blog post and extract its SEO content). No change to the client sites. The plan is the current direction and can change.
Repository	seo-advisor (new repository). It reuses tested parts of the SEOAdvisor prototype.
Example sites	Test cases, not targets: MyPipit (marketplace.mypipit.com , travel, English, 58 URLs on 7 October 2026) first, blog posts first. extendmy.life (extendmy.life , longevity and health, English and German, about 474 URLs) later.
Decisions so far	Single page first, then the whole site (8 Oct 2026). AI suggestions with DeepSeek. Read-only access to the sites. Gurzu SEO team reviews first, then the client approves. A person saves every change. Drafts in German need native review. Health-content review rule: decide before extendmy.life drafts start.
Evidence	Google Search Central, Search Quality Rater Guidelines (11 Sep 2025), PostgreSQL, SQLite, DBOS, AWS and Microsoft architecture guidance, schema.org, web.dev, GOV.UK, RFC 9309, OWASP, NIST, DORA, GitHub, Anthropic. See sections 24 and 25.

How to read the diagrams



1. Summary

What we build

seo-advisor is one platform for many sites. It starts with one page. A person gives the URL of a page on a registered site and a target keyword. The tool fetches that page, extracts its SEO content and audits it. Each finding shows its evidence and its source. Then an AI suggests fixes, and a person reviews each suggestion. Later the tool scales to the whole site: an inventory of all pages, scheduled audits, a work queue, and results measured in Search Console.

Main decisions

Topic	Decision	Reason
Scope	Single page first. The person pastes a URL of a registered site and types a keyword. The sitemap list is only a page picker. The whole site comes in Phase 3.	Owner decision (8 Oct 2026): do SEO on one page before the whole site.
Access to client sites	Read-only. A person copies approved text into the site editor and saves. An extension fills the editor in Phase 4.	No client code change needed. People stay in control.
AI provider	DeepSeek writes suggestions. A person reviews each one.	Owner decision (8 Oct 2026). The AI only writes text. Code counts and scores.
Database	PostgreSQL 18 with pgvector	Many writers at the same time (scheduler, workers, API). SQLite allows one writer (S20).
Jobs and schedules	On request in Phase 1. DBOS on the same PostgreSQL from Phase 3.	Durable steps, schedules per site, retries, no extra server (S22).
Tenancy	One shared schema with tenant_id and site_id on every row	Lowest cost and effort. Add row-level security when outside customers arrive (S23, S24).
Fetching	Plain HTTP (httpx) through the safe fetcher, one page at a time. A full crawl in Phase 3.	Both sites send full content in the first HTML response (observed 5 Oct 2026).
Reviews	Gurzu second-person review (2i), then client approval. Expert review for health and native review for German.	Google: health content must be accurate and written or reviewed by an expert (S2, S3).
Tools and setup	uv, pnpm, mise, Docker Compose, strict automatic checks, light process rules	One setup command. New engineers can start fast (sections 15 to 19).

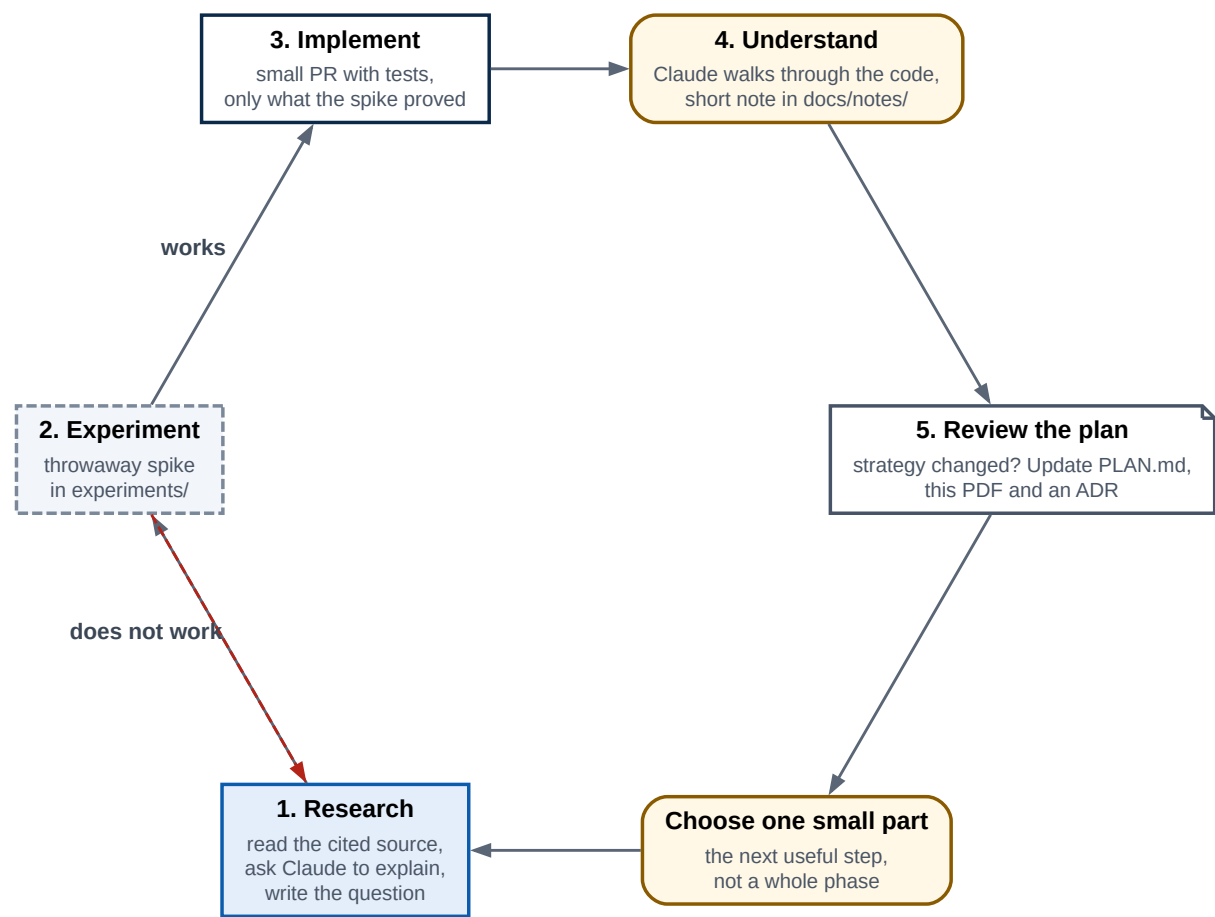
How to begin

1. Read this document and `PLAN.md`. Agree on the open questions in section 23.
2. Setup steps 0.1 to 0.3 are done. The other setup steps start when a slice needs them (sections 16 to 18).
3. Build the page loop for one MyPipit blog post: fetch and extract (slice 2), audit (slice 3), AI suggestions (slice 4). In each slice, build the backend first, then its screen. Do not build AI suggestions before the audit results are correct.

WARNING: Do not publish AI-written health content without an expert review. Google's rater guidelines rate an AI-written medical article made at scale as "Lowest: Scaled content abuse" (S1).

How we work: research, experiment, implement

This project follows a cycle of research, experiment and implementation. Each small part goes through the cycle before the next part starts.



Rule	What it means
Start small	Build one small part at a time. Make it bigger later. Do not implement a whole phase at once.
Research first	Read the source that the task names. Ask Claude Code to explain it. Write down the question that the part must answer.
Experiment before code	Make a small throwaway test in the experiments folder. Keep what works. Write down what does not work.
Implement only what is proved	Make a small pull request with tests. It contains only what the experiment proved.
Understand before you continue	Ask Claude Code to walk through the new code until you can explain it yourself. Write a short note in docs/notes. Then choose the next part.
A general tool	MyPipit and extendmy.life are example sites for tests. Do not shape the design around them.
The plan can change	The phases are the current direction, not a contract. When a strategy changes, update PLAN.md, this PDF and an ADR. Strike out the replaced tasks and give the reason in one line.

2. The two sites today

Facts from the public pages, read with GET requests only, on 2, 5 and 7 October 2026.

Check	MyPipit (marketplace.mypipit.com)	extendmy.life
Type	Travel marketplace (treks, hotels, guides)	Longevity and health marketplace (clinics, retreats, articles). YMYL topic (S1).
Size	58 URLs on 7 October 2026: 30 blog posts, 17 listings, 11 other pages (53 URLs on 2 October)	About 474 URLs: 82 articles, 125 clinic pages, 3 categories, 11 subcategories, 13 groupings, 3 static pages, each in EN and DE
Technology	Rails, server-rendered pages	Next.js, server-rendered pages
Languages	English	English and German with reciprocal hreflang (en, de, x-default) on inner pages
Good base	Canonical, meta description, Open Graph, sitemap, BlogPosting data	Canonical, good titles on articles and clinics, alt text on images, robots.txt with sitemap
Main issues	Titles without the searched trek name; image links expire after 1 hour; 41 of 88 images without alt text; no robots.txt	Same meta description on all category pages; subcategory pages repeat the parent H1; articles have empty author and no dates in structured data; all sitemap lastmod values are the build time; home page has no canonical or hreflang; x-default redirects (307)
Content editor	Rails agency dashboard (Lexxy rich text)	Not known yet (question in section 23)
Search Console	MyPipit team has access	Not confirmed
Order	First. Blog posts first, one page at a time.	Later (Phase 5).

3. Requirements

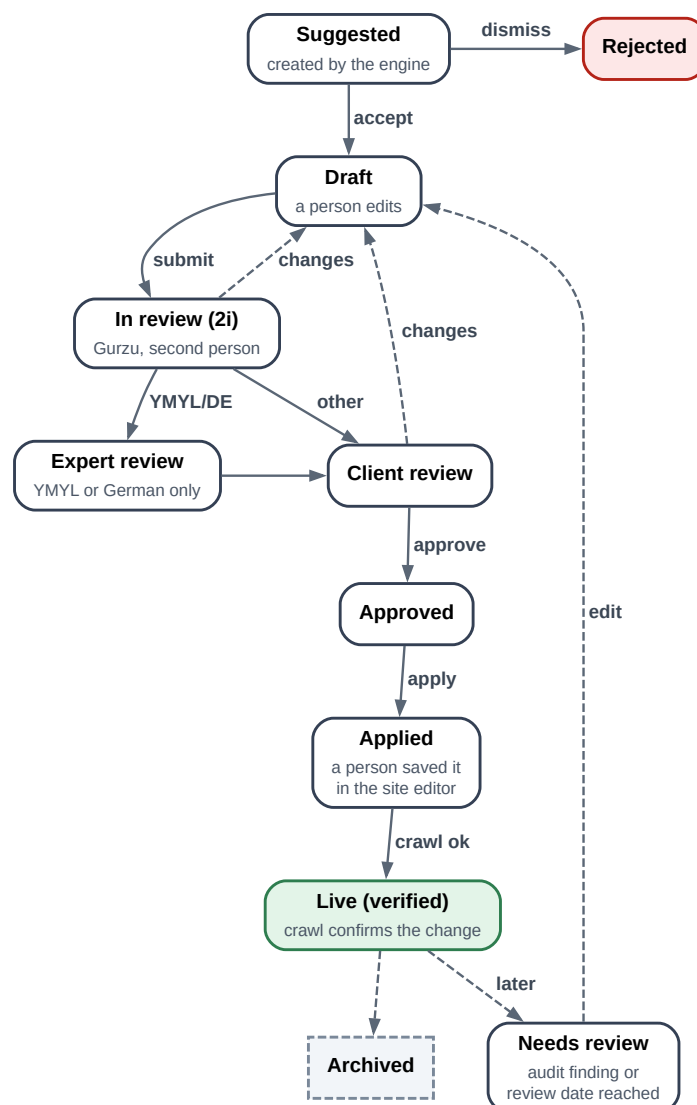
FR-1 to FR-10 make the page loop of Phase 1. FR-11 to FR-16 come when the tool scales.

ID	Requirement	Priority
FR-1	Register a site: base URL, language, page-type rules.	Must
FR-2	A person gives one page URL and a target keyword. The tool accepts only URLs on a registered site.	Must
FR-3	Fetch that one page read-only through the safe fetcher (SSRF guard, robots.txt, rate limit). Keep a snapshot.	Must
FR-4	Extract the SEO content: title, meta description, canonical, meta robots, lang, H1 to H6, images and alt text, links and anchor text, Open Graph, structured data, main text and word count.	Must
FR-5	Audit the page with the rules for its page type (blog posts first). Each finding has severity, evidence (found, expected, where) and a source. A check that cannot see its data says "unknown", not pass.	Must
FR-6	Check the page again after a change. Compare with the last run: new, fixed and returning findings.	Must
FR-7	AI suggestions for the title, meta description, H1, alt text and content notes. Facts only from the page; other facts are [ADD: ...].	Should (slice 4)
FR-8	Checks on every suggestion: no invented numbers or names, no copied text.	Must (with FR-7)
FR-9	A person accepts, edits or rejects each suggestion and copies it into the site editor. The tool never publishes.	Must (with FR-7)
FR-10	Sitemap list as a page picker: read robots.txt and the sitemaps, show the URLs with their page type.	Should
FR-11	Page inventory of the whole site; crawler; audits daily, weekly and monthly per site.	Later (Phase 3)
FR-12	Search Console data every day; keyword-to-page map with one primary keyword per page and language; work queue ranked by effect and effort.	Later (Phase 3)
FR-13	Content items with workflow states, roles, versions and events.	Later (Phase 3)
FR-14	Chrome extension: show approved suggestions in the site editor and fill fields when a person accepts. It never saves.	Later (Phase 4)
FR-15	Language versions (EN-DE, hreflang groups) and health rules.	Later (Phase 5)
FR-16	Measure each change at 4, 8 and 12 weeks: per page first, then against unchanged pages of the same type. Weekly report and alerts.	Later (Phase 6)
NFR-1	Read-only on client sites. Respect robots.txt (RFC 9309) and rate limits per host.	Must
NFR-2	Every number shows its source and date. Every AI call is logged with model, prompt version and cost.	Must
NFR-3	Multi-site and multi-tenant data model from day 1 (tenant_id, site_id on every row).	Must
NFR-4	A failed source does not stop a run; the run records the gap.	Must
NFR-5	Cost per site per month is tracked and capped.	Should
NFR-6	Tests run without paid APIs (recorded data). CI on every pull request.	Must

4. Users, roles and approvals

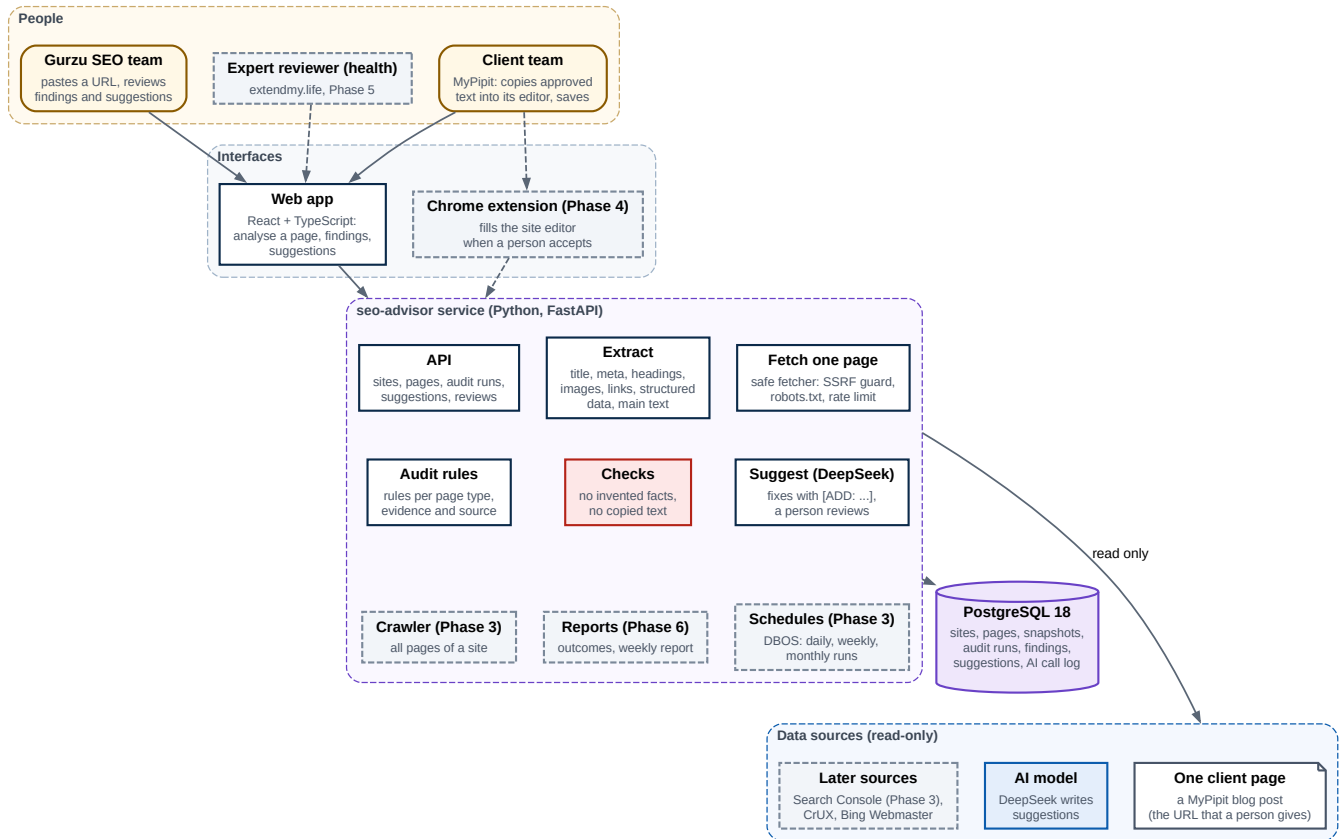
Nobody approves their own work. In Phase 1 a person accepts, edits or rejects each suggestion. The full states below come with the content workflow (Phase 3). Health (YMYL) and German content need an extra expert or native review.

Role	Who	Can do	Cannot do
Admin	Gurzu lead	Add sites, users, schedules, rules	Approve own drafts for YMYL items
SEO editor	Gurzu SEO team	Edit drafts, run audits, submit for review, do the 2i review of another person's draft	Review own draft
Expert reviewer	Medical expert (health) or native German speaker	Approve or reject the facts or the language	Change workflow settings
Client approver	MyPipit or extendmy.life team	Final approval; apply and save in the site editor	Change audit rules
Viewer	Anyone invited	Read reports	Edit



5. Architecture

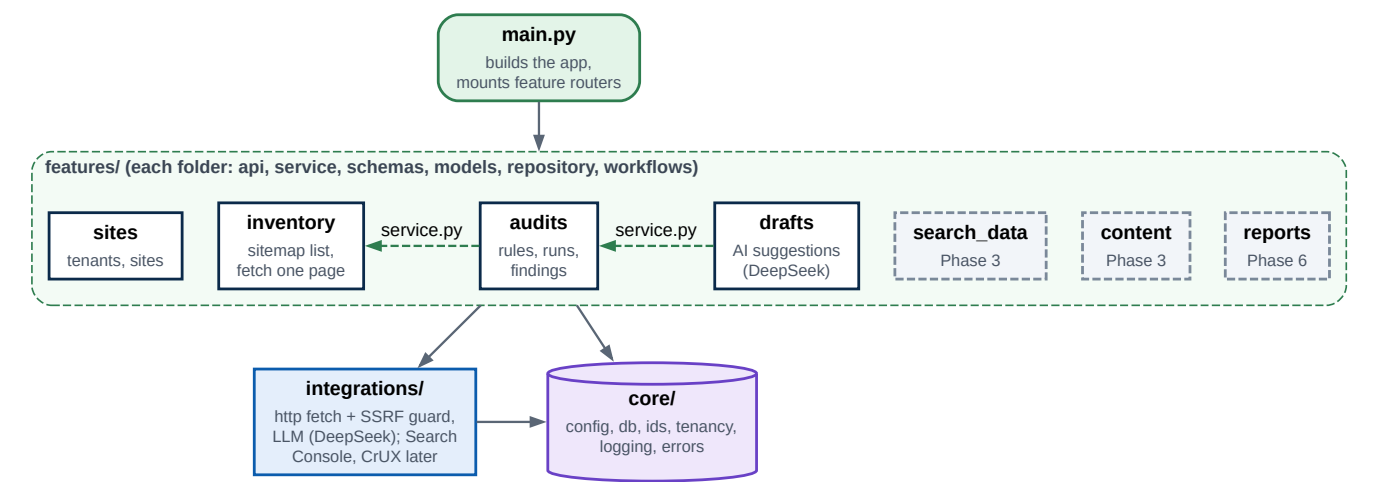
Layers: people → interfaces → one Python service with clear modules → PostgreSQL → read-only data sources.
Dashed boxes come later.



NOTE: One service (a modular monolith) is enough for 1 to 2 engineers. Each module has a clear interface, so a module can move to its own service later. In Phase 1 the service works on one page at a time, on request. The dashed boxes come in later phases. The extension holds no logic and no keys; Chrome bans remotely hosted code (MyPipit evidence S17).

5. Architecture (continued): code grouped by feature

The platform is one service (a modular monolith). The code is grouped by feature, not by type. All code for one feature is in one folder.

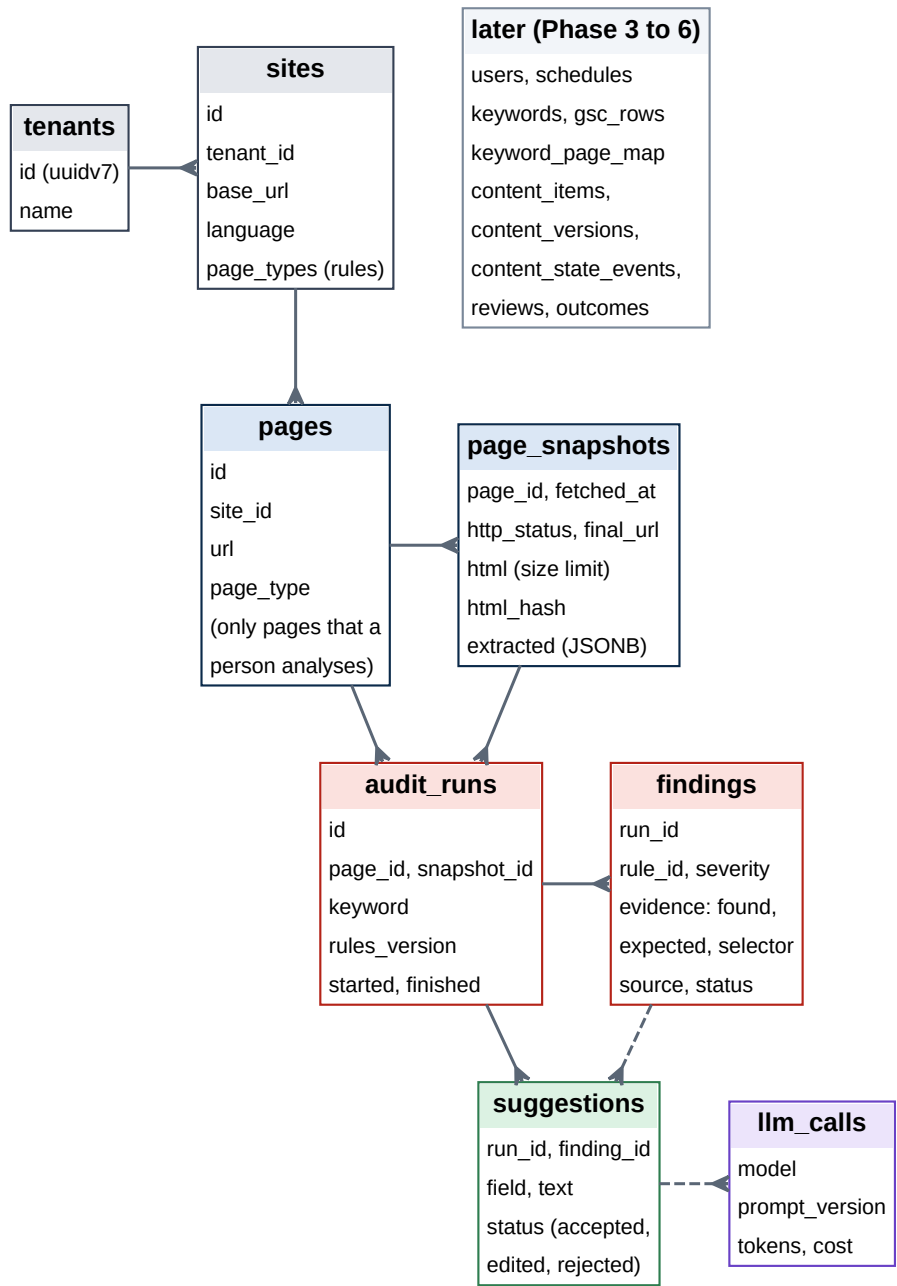


```
apps/api/src/seo_advisor/
core/          config, db session, ids (uuidv7), tenancy, logging, errors (RFC 9457)
integrations/  external clients only: http_fetch (+ SSRF guard), search_console, crux, llm
features/
  sites/  inventory/  audits/  drafts/          (Phase 1: one page)
  search_data/  content/  reports/          (later phases)
  api.py  service.py  schemas.py  models.py  repository.py  workflows.py  prompts/
main.py    builds the app and mounts each feature router
apps/api/tests/<feature>/      apps/web/src/features/<feature>/  (same features)
```

Rule	Reason
A feature imports another feature only through its service.py. It never imports their models or repository.	Each feature can change inside without breaking others. import-linter protected contracts check this rule in CI (S45).
core/ and integrations/ import no feature.	Shared code stays small and has no hidden links to features.
Vendor APIs are called only in integrations/.	A vendor change touches one folder (same rule as SEOAdvisor).
A new feature is a new folder plus one line in main.py. Its tables go into the one shared migration history.	Work stays in one place. A new engineer finds all code for a feature in one folder.
The web app uses the same feature names.	One name for one thing, from database to screen.

6. Data model

Every table has `tenant_id` and `site_id` (not all shown). Crow-foot arrows mean one-to-many. Header colours: grey = tenancy, blue = page and snapshot, red = audits, green = suggestions, purple = AI call log. The grey box lists the tables of later phases.



NOTE: IDs use `uuidv7()` (PostgreSQL 18), so rows sort by time. In Phase 1 the table `pages` holds only the pages that a person analyses, not the whole sitemap. Each check of a page adds a snapshot and an audit run, so two runs can be compared. Suggestions and AI call logs are never deleted.

7. Database: options we compared

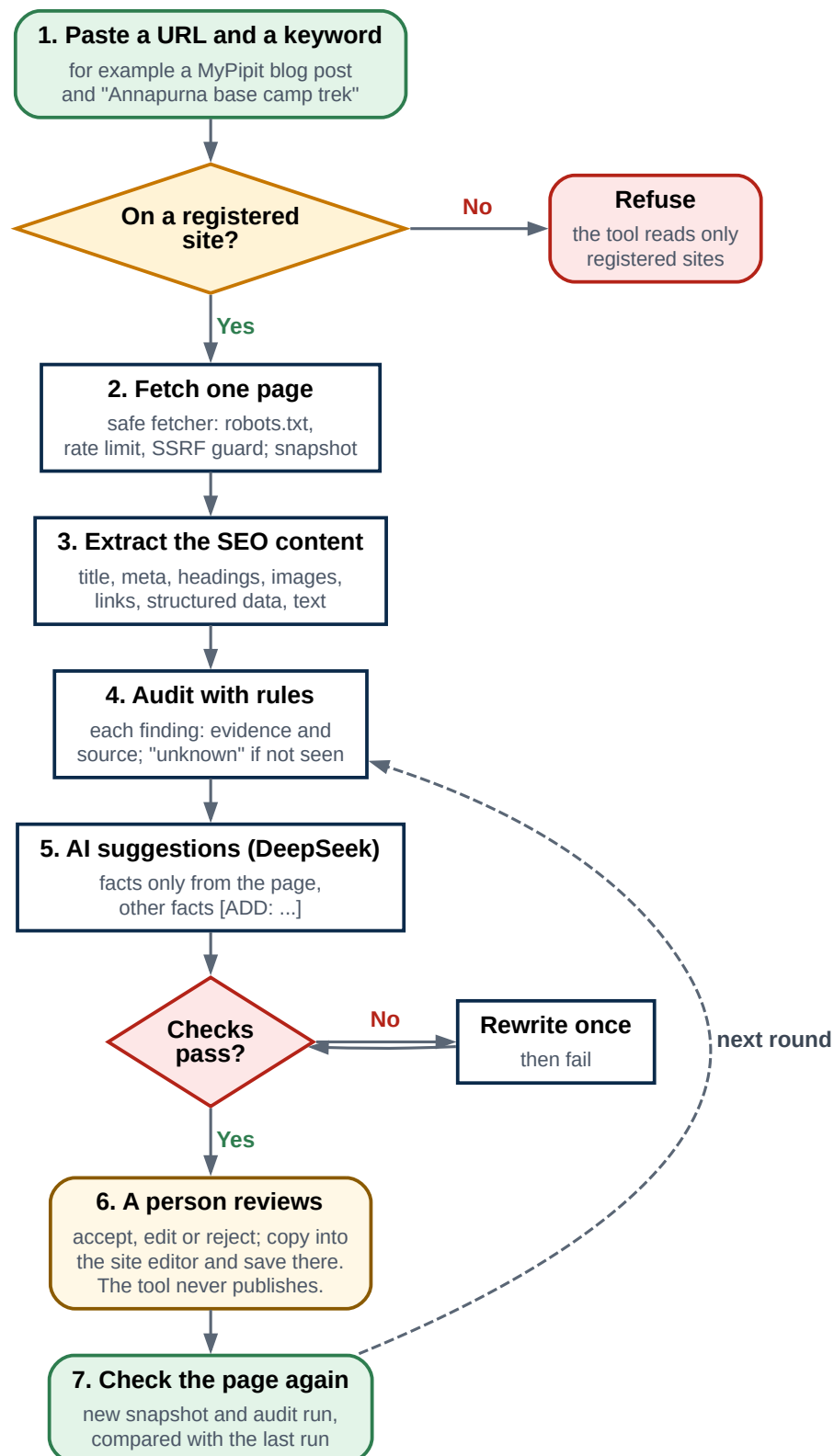
The platform has many writers at the same time. This decides the database.

Option	Benefit	Drawback	Decision
PostgreSQL 18 + pgvector	Many writers; JSONB; full-text search with English and German stemmers; row-level security; partitioning; vectors in the same database (S20).	A server to run. Row-level security needs care: table owners skip it unless forced.	Chosen
SQLite	No server; simple backups.	One writer at a time, also in WAL mode. SQLite's own guide says to use a client/server database for many concurrent writers.	Rejected
MongoDB	Flexible documents.	Our data is relational (sites → pages → items → versions). SSPL licence.	Rejected
TimescaleDB	Time-series helpers.	Key features are under a licence that forbids selling as a service. Plain PostgreSQL is enough at this size.	Rejected
Separate vector database	Fast vector search at large scale.	One more system. pgvector is enough for thousands of pages.	Later

PostgreSQL detail	Rule for seo-advisor
pgvector index	HNSW for search speed. Indexed vectors are limited to 2,000 dimensions; ask the embedding model for a smaller size or use halfvec.
Full-text search	Use the english and german configurations. German compound words need an extra dictionary later.
Partitioning	Not now. The docs say it pays off when a table is larger than server memory. Add monthly partitions to gsc_rows later.
Row-level security	Not in phase 0. The app filters by tenant_id. When outside customers arrive, enable RLS with FORCE and a non-owner app role.

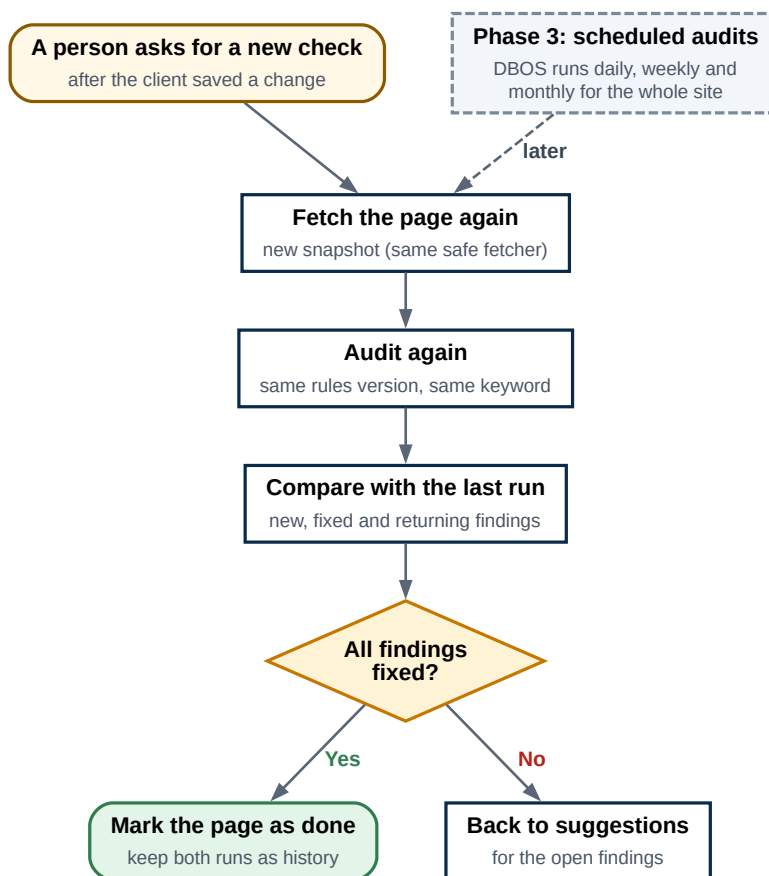
8. Flow A: analyse one page

The page loop of Phase 1. A person starts it for one page. Nothing runs on a schedule.



9. Flow B: check the page again

In Phase 1 a person asks for a new check after a change. Each check compares with the previous run, so the team sees what is new, fixed or back. Schedules come in Phase 3.

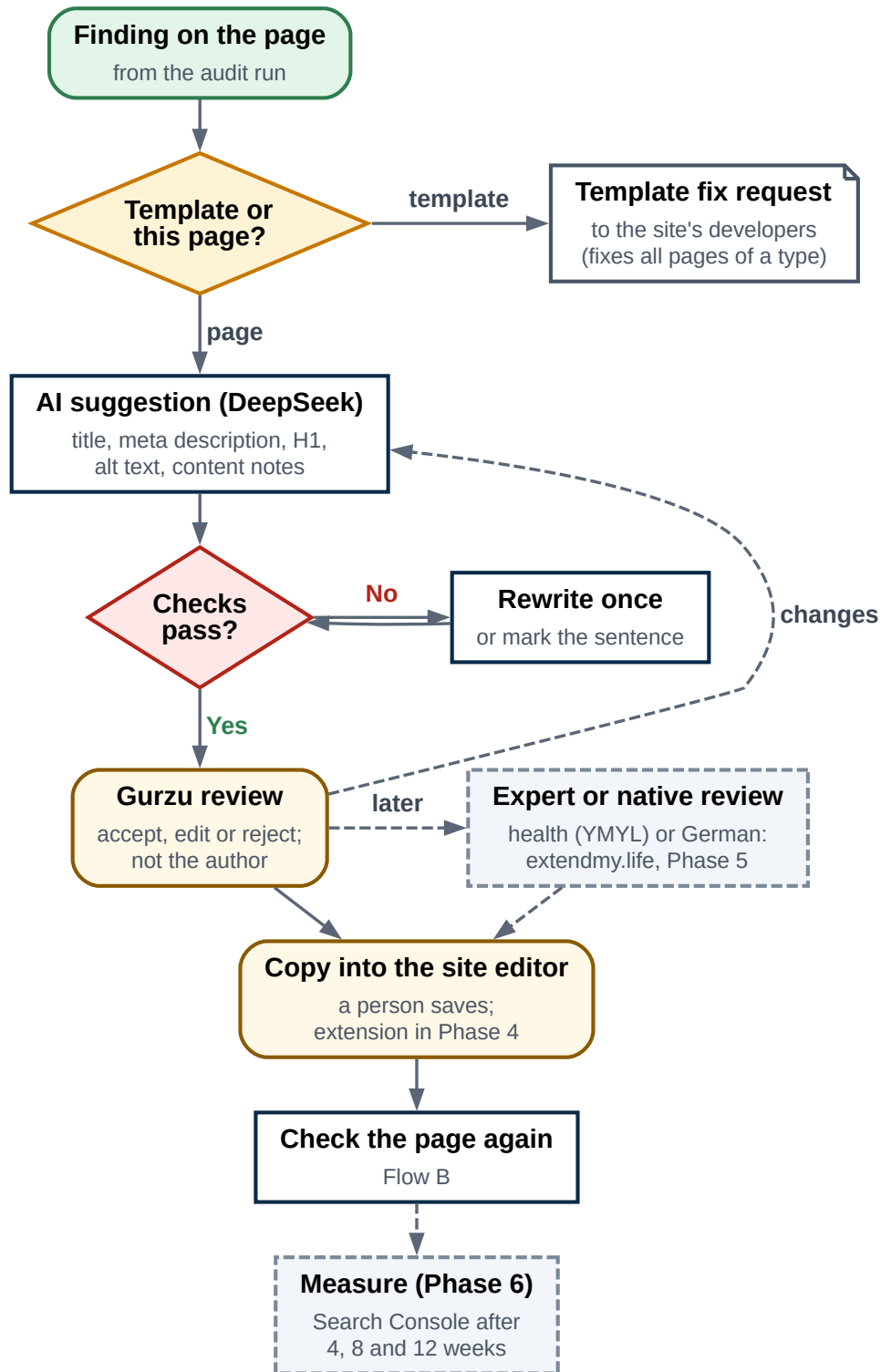


Later (Phase 3): scheduled audits for the whole site

Run	What	Why
Daily	robots.txt, sitemaps, status codes, new and removed URLs, Search Console sync	Errors must show fast. Search Console data is 2 to 3 days late.
Weekly	All rules on pages that changed and on a rotating sample	Stable view without crawling everything every day.
Monthly	Full audit, keyword map refresh, review dates, Core Web Vitals (CrUX API)	Content and keywords change slowly. CrUX data covers 28 days.

10. Flow C: from finding to fix

A finding on the page becomes a suggestion. A template issue goes to the site developers. A page issue gets an AI suggestion that passes checks and a human review. A person copies it into the site editor and saves it. A finding on the page becomes a suggestion. A template issue goes to the site developers. A page issue gets an AI suggestion that passes checks and a human review. A person copies it into the site editor and saves it.



11. Managing many content items (Phase 3: whole site)

This section is for Phase 3, when the tool scales to the whole site. Phase 1 works on one page at a time. extendmy.life alone has about 474 URLs. The platform must manage this volume without a person opening every page. These are the rules.

Rule	How it works	Example
One inventory	Every page, field and language version is a row. Nothing is edited outside the platform.	The German and English versions of one article share one hreflang group.
Page types and templates	Rules run per page type. A finding on many pages of one type is a template fix, not 100 single fixes.	One meta description on all 17 category pages → one request to the developers.
Priority score	Impact (Search Console impressions, page type value) × confidence ÷ effort. The queue shows the top items first.	A clinic page with 3,000 impressions at position 8 comes before a page with 20 impressions.
Batches	Work in batches of 5 to 10 items of the same type, so reviewers keep one context.	Batch: 8 retreat pages, titles and descriptions only.
Keyword map	One primary keyword per page and language. The platform blocks a second page with the same primary keyword.	'longevity clinic poland' belongs to one clinic page only.
Review dates	Each page gets a next review date. Health pages need a shorter cycle. The audit sets 'Needs review' when the date passes.	Health article: review every 6 months (example, to agree).
Major and minor changes	Only a major change (new facts, prices, advice) updates dateModified and the sitemap lastmod.	Fixing a typo is minor: dates stay.
Bulk actions with care	Bulk changes are allowed only for structure (for example alt text drafts). Each item still needs review.	40 alt text drafts reviewed in one screen.

CAUTION: Do not generate many pages from one template without real added value. Google lists this as scaled content abuse (MyPipit evidence S12).

12. Audit rules

Each rule records its source. A rule that is only common practice or a rule of thumb says so. R3 is the MyPipit research file (requirements/research/03-mypipit-evidence.md).

Phase 1: first rules for a blog post

Area	Rule	Source
Title	Present; fits about 600 pixels (rule of thumb; reuse the SEOAdvisor pixel check); the target keyword is in the title	R3 §1
Meta description	Present and unique; 120 to 156 characters is a rule of thumb, not a Google rule	R3 §2, §11
Headings	Exactly one H1; the keyword is in the H1; heading levels in order	Practice
Images	Alt text present, descriptive and not stuffed; image URLs are stable (MyPipit image links expire after 1 hour)	R3 §3.1, §3.2
Social image	og:image present, at least 1200 x 630 pixels	R3 §3.3
Indexing	Canonical present and self-referencing; no noindex	R3 §7
Structured data	BlogPosting with author, datePublished, dateModified, headline and image	R3 §5.11
Links	Internal links are <a href> links with descriptive anchor text	R3 §13.1
Slug	Readable words with hyphens, no IDs	R3 §8.1
Page	lang attribute present; word count shown as information only (Google has no preferred word count)	R3 §6.4

Later: rules for the whole site, other page types and other sites

Group	Rule	Source
Technical	HTTP status 200; no redirect chains; HTML under 2 MB (Googlebot reads the first 2 MB)	Google (S9)
Technical	robots.txt valid; Sitemap line present; parameter rules block all positions of filter parameters	RFC 9309 (S10), Google faceted navigation (S8)
Technical	Sitemap lastmod changes only when the page content changes	Google (S7), Bing (S11)
Language	hreflang links reciprocal, full URLs, same-language canonical, x-default to a real page	Google (S5)
Language	Visible text is in the declared language; German text reviewed by a native speaker	Google (S5)
On-page	Meta description unique across the site (no shared category text)	Google
Structured data	Clinic pages: MedicalClinic (a LocalBusiness type) with name and address; no self-serving review stars	schema.org, Google (S4)
Health (YMYL)	Named author or reviewer with real credentials; visible last reviewed date; no invented experts	Google (S2, S3)
Content	Covers the basics for its page type; answers the main user questions; no keyword stuffing	Google helpful content (S2)
Freshness	Review date not passed; dates change only with major changes	Google (S2), GOV.UK (S16)
Speed	LCP ≤ 2.5 s, INP ≤ 200 ms, CLS ≤ 0.1 at the 75th percentile (when CrUX has data)	web.dev (S12)

13. Content generation rules

The same rules apply to every site. Health sites add expert review and stricter checks.

Rule	Detail
Facts only from the page	The draft uses facts that are in the current page or in data that the client gave. Every other fact is an [ADD: ...] placeholder.
No copied text	No run of 8 words may match another site's page.
Second model checks	A model from another family checks that each sentence has support in the source.
German drafts	The platform drafts German text, but a native German speaker must review it before approval. Machine translation alone is not enough (S6).
Health (YMYL) drafts	Rule to decide before extendmy.life drafts start: a named expert reviewer, or AI limited to non-medical text. Never invent authors or credentials (S3).
Disclosure	For health content, show who wrote and reviewed it, and how it was made (Who, How, Why) (S2).
Legal check (Germany)	German advertising law (HWG §11) limits before/after images and testimonials for cosmetic procedures. A lawyer must confirm how it applies to aesthetic clinic pages.
No word-count targets	Google has no preferred word count. The draft covers what the user needs.
People publish	The extension fills the fields. A person clicks Save. The platform never publishes.

14. Options we compared

Topic	Option	Benefit	Drawback	Decision
Site access	Read-only + extension	No client code change; people stay in control	The extension depends on the editor layout	Chosen
	Write through site APIs	Fully automatic	Needs developer access and trust on both sites	Later
	Export packs only	Simple	Manual copying, errors	Rejected
Structure	Modular monolith	One deploy, simple for 1 to 2 engineers	Must keep module borders clean	Chosen
	Microservices	Independent scaling	Too much operations work now	Rejected
Code grouping	By feature (sites, inventory, audits ...)	All code for one feature in one folder. Small, clear changes	Needs a rule check for imports between features	Chosen
	By type (all models, all services, all routes)	Familiar	One change touches many folders. Features mix	Rejected
Jobs	DBOS on PostgreSQL	Durable steps, cron per site, no Redis	Newer library	Chosen
	Procrastinate	Simple PostgreSQL queue	Schedules need own table	Later
	Celery + Redis	Well known	Extra Redis; one scheduler process only	Rejected
Tenancy	Shared schema + tenant_id	Lowest cost and effort	Isolation in the app until RLS	Chosen
	Schema per tenant	Some isolation	Every migration N times	Rejected
Crawler	httpx from sitemaps	Fast, cheap, reuses SEOAdvisor code	Not for JavaScript-only pages	Chosen
	Headless browser for all	Sees rendered pages	Slow, heavy; not needed for these sites	Later
Repo	New repo, reuse modules	Clean multi-site design	Some copying	Chosen

15. Technology stack

The versions are the current versions on 6 October 2026. Check them again on the day of the setup.

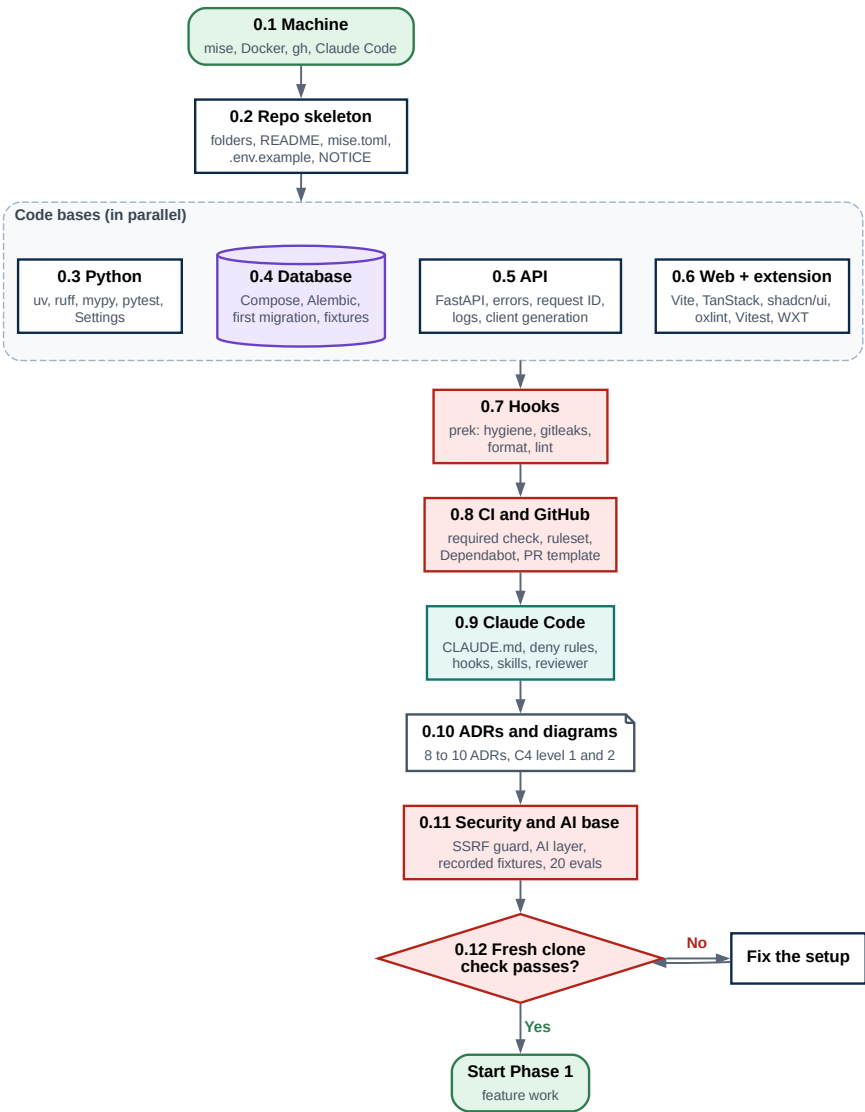
Layer	Choice (version on 6 Oct 2026)	Reason
Code structure	Modular monolith grouped by feature, import-linter contracts	Easy to work in one feature at a time (section 5, continued).
Tools and tasks	mise 2026.10 pins Python, Node, uv, pnpm and prek. It also runs the project tasks.	One file gives every person and CI the same tools and commands (S31).
Python	Python 3.14.8, uv 0.12 (lock file uv.lock). Package name seo_advisor (not seo_engine, which SEOAdvisor uses).	3.15 is new and DBOS, FastAPI and Pydantic do not support it yet (S26). uv replaces pip, venv and pip-tools (S25).
API	FastAPI 0.142, Pydantic 2.13, pydantic-settings 2.15	Typed inputs and outputs. OpenAPI generates the TypeScript client.
Database	PostgreSQL 18 + pgvector 0.8.7, SQLAlchemy 2.1 (sync), psycopg 3, Alembic 1.20	Sync code is simpler for new engineers and fits DBOS (S39).
Jobs	DBOS 3.2 on the same PostgreSQL, from Phase 3. Phase 1 runs on request.	Durable steps and schedules without a second server (S22).
Quality (Python)	ruff 0.16 (lint and format), mypy 2.4 --strict, pytest 9.1 strict, testcontainers 4.15	One type checker as the gate. Tests use a real database (S27, S28).
Web app	React 19.3, TypeScript 6.0.3, Vite 8.3, pnpm 12, TanStack Router, Query and Table, Tailwind CSS + shadcn/ui, oxlint + Prettier, Vitest 5. React Hook Form + Zod when the first real form needs them. Recharts from Phase 6	TypeScript 7 has no tool API until 7.1. The Vite template uses oxlint (S29).
API client	openapi-typescript 7.13 + openapi-fetch 0.17	Small and stable. CI fails if the client is out of date.
Extension	WXT 0.21 (Manifest V3)	Less custom build code. Shares Vite, React and TypeScript with the web app (S30).
Logs and traces	structlog 26.1, OpenTelemetry 1.45 (later step)	Logs with request ID from day 1 (S38).
AI	DeepSeek writes suggestions (owner decision, 8 Oct 2026). A checker from another model family later.	Prompts as versioned files. Every call logged with cost (S41).
Docs	playwright 1.63 (Python) and @hpcc-js/wasm-graphviz 1.29 (npm), docs only	mise run docs:pdf builds this PDF with the installed Google Chrome.
Git and CI	GitHub Actions, Dependabot, gitleaks, prek hooks	Free tools. Paid GitHub security later (S33).
Deployment	Later, with DevOps	Not in this plan.

All pinned versions (6 October 2026)

Python 3.14.8 · uv 0.12.23 · ruff 0.16.10 · mypy 2.4.0 · pytest 9.1.1 · testcontainers 4.15.0 · FastAPI 0.142.2 · Pydantic 2.13.5 · pydantic-settings 2.15.0 · SQLAlchemy 2.1.3 · psycopg 3.3.6 · Alembic 1.20.0 · DBOS 3.2.0 · structlog 26.1.0 · pytest-recording 0.14.0 · Node 24 LTS (Node 26 LTS from 28 October 2026) · pnpm 12.9 · Vite 8.3.3 · React 19.3.0 · TypeScript 6.0.3 · oxlint 1.87.0 · Prettier 3.9.9 · Vitest 5.0.3 · Playwright 1.63.0 · WXT 0.21.4 · openapi-typescript 7.13.0 · openapi-fetch 0.17.0 · mise 2026.10.3 · prek 0.5.5 · gitleaks 8.30.1 · pip-audit 2.10.1 · Docker image pgvector/pgvector:0.8.7-pg18-trixie · import-linter 2.15

16. Set up the repo

Steps 0.1 to 0.3 came first. Since 7 October 2026 the other steps start when a slice needs them, so feature work and setup go together.



Principle	What it means
One command to start	A new engineer runs <code>mise run setup</code> and then <code>mise run check</code> (or <code>make check</code>). Nothing else is necessary.
Automatic checks, light process	Lint, format, types and tests run by themselves. Hooks only fix files or block secrets. Process rules stay few, so new engineers can work fast.
Decisions in writing	A decision record (ADR) for each main choice. People and AI tools read the reason for each choice (S35).
Same environment for all	<code>mise</code> pins the tools. Docker Compose runs the database. Lock files fix every package version.
Safe by default	No secrets in git. Actions pinned to commit IDs. The fetcher blocks internal addresses (S33, S37).

16. Set up the repo (continued): when each step starts

Feature-first (7 Oct 2026): each setup step starts when a slice needs it. Sections 17 and 18 give the full task and the "done when" of each step.

Step	Starts with
0.1 to 0.3	Done.
0.4 Database	Slice 2. Only the tables that each slice needs: tenants, sites (done); pages, page_snapshots (slice 2); audit_runs, findings (slice 3); suggestions, llm_calls (slice 4).
0.5 API	Started 8 Oct 2026 with the web app; structlog is still open.
0.6 Web app	Started 8 Oct 2026: each slice now ends with its screen. The extension starts in Phase 4.
0.7 Hooks, 0.8 CI	Before the first pull request into dev, or when the owner decides.
0.9 Claude Code settings	When the owner decides. Recommended soon: deny reads of .env.
0.10 ADRs	Write each ADR when its decision is made or first used.
0.11 SSRF guard	Slice 1 (the first outbound fetch). Done.
0.11 AI layer, evals	Slice 4 (AI suggestions, DeepSeek).
0.12 Final check	Before a second person joins.

17. Setup checklist, part 1

The detail and the sources for each step are in research file R4 (requirements/research/04-project-setup.md). Section 16 says when each step starts.

Step	What to set up	Done when
0.1 Machine Done	Install mise, Docker, the GitHub CLI and Claude Code. Set the git identity raksha-gurzu. Push through the github.com-gurzu SSH alias.	The three tools report their versions and gh is logged in.
0.2 Repo skeleton Done	Folders apps/api (with core, integrations and empty feature folders), apps/web, apps/extension, packages/api-client, docs (with decisions, architecture, design and notes), evals, experiments, infra, .github. Copy this requirements folder into the repo. README with a 5-minute setup, .gitignore, .editorconfig, .env.example with fake values, a proprietary NOTICE. mise.toml with tool versions and tasks (setup, dev, test, lint, typecheck, check, audit, db:up, db:migrate, db:seed, gen-client, evals, docs:pdf). The docs:pdf task rebuilds this PDF.	A new person follows the README and <code>mise run setup</code> works on a fresh clone.
0.3 Python base Done	pyproject.toml, uv.lock, dependency groups (dev, test, lint), a 7-day delay for new package releases (exclude-newer), src layout. ruff rules E F W I U P B S I M S A S Y N C P T R U F D T Z N. mypy --strict with the Pydantic plugin. pytest strict mode and markers (unit, integration, e2e, live). import-linter protected and forbidden contracts for the feature rules. (Do not use the independence contract. It also blocks the allowed calls through service.py.) A vulnerability check task (<code>mise run audit</code> , <code>pip-audit</code>). A Settings class that reads all configuration from the environment and stops at start if a key is missing.	<code>mise run check</code> passes. A forbidden import between features fails the check. A missing key stops start-up with a clear message.
0.4 Database base Started	Docker Compose with the pgvector image. Mount the volume at /var/lib/postgresql (PostgreSQL 18 changed this path, S32). SQLAlchemy base with the Alembic naming convention. First migration: vector extension, uuidv7() keys, timestamps with time zone, first tables. Test fixtures: testcontainers starts one test database for each session, rollback after each test, DBOS reset. Seed script with 2 fake sites.	Upgrade and <code>alembic check</code> pass on an empty database. An integration test passes twice in a row.
0.5 API base Started	FastAPI under /api/v1 with a health route. An error handler for RFC 9457 problem details (FastAPI has none, S40). Request ID middleware. structlog. An operation ID on each route. A task that exports openapi.json and generates the TypeScript client.	An error returns application/problem+json with the request ID. The web app calls /health through the generated client.
0.6 Web and extension base Web done	Vite react-ts template with strict TypeScript flags, oxlint, Prettier, Vitest, Testing Library and MSW. TanStack Router, Query and Table. Tailwind CSS and shadcn/ui, with the components in src/shared/ui. Folders app, routes, features and shared, as in CLAUDE.md. Check the version, licence and advisories of each library that R4 does not cover. Keep the pnpm release delay. Allow no install scripts until a package needs one. WXT extension skeleton with minimal permissions and no remote code.	Lint, type check, tests and build pass. The extension loads in Chromium and calls /health.

18. Setup checklist, part 2

Step	What to set up	Done when
0.7 Hooks	One .pre-commit-config.yaml, run by prek: file hygiene, gitleaks, ruff fix and format, oxlint and Prettier, lock file check, actionlint, zizmor.	Hooks finish in less than 5 seconds. A planted fake key is blocked.
0.8 CI and GitHub	One CI workflow. Pin actions to commit IDs and turn on the GitHub policy that requires pinned actions. Read-only token by default. Jobs: lint, types, tests, migrations, OpenAPI drift, web, extension, gitleaks, and one required job. Ruleset on main: pull request, required check, squash merge only, linear history, no force push. 0 approvals while one engineer works, 1 approval when a second engineer joins. Check the pull request title for Conventional Commits. Dependabot each week for uv, pnpm, actions and Docker. A pull request template with an AI-assisted line.	A failing test blocks the merge. A direct push to main is refused. The first Dependabot pull request passes CI.
0.9 Claude Code	CLAUDE.md is at the repo root (under 200 lines). .claude/settings.json: deny reads of .env files, keys, lock files, build output, node_modules, the generated client and PDFs, deny force push, allow the project commands, a format hook after edits, fast tests when Claude stops, sandbox on. Skills: new-endpoint, new-migration, run-evals. A security-reviewer subagent.	Claude cannot read .env. Claude runs mise run check from CLAUDE.md alone.
0.10 Decisions and diagrams	8 to 10 ADRs in MADR 4.0 format (layout, Python and uv, PostgreSQL and pgvector, DBOS, sync SQLAlchemy, code-first OpenAPI, AI providers, read-only rule, WXT, logging). C4 level 1 and 2 diagrams. A design document template.	Each ADR has context, options, decision and consequences.
0.11 Security and AI base Guard done	SSRF guard with tests for each blocked address range and for DNS rebinding (reuse from SEOAdvisor). AI layer at slice 4 (DeepSeek; reuse SEOAdvisor providers/llm.py): provider interface, model and price settings, llm_calls table, prompts as versioned files, recorded test fixtures with keys removed, cost limits per run and per day. One key for each person, and a spend limit at each provider. evals folder with 20 first cases and code-based graders.	Guard tests pass before crawler code exists. A recorded AI call replays in CI with no network. mise run evals shows a score table.
0.12 Final check	Fresh clone, setup, check, CI, secret block and push block, all from the README only.	A second person or a new Claude session completes it in 15 minutes or less.

Should do in weeks 2 to 4

SSH commit signing. Issue forms, labels and milestones for each phase. Semgrep and a weekly dependency audit. A coverage floor that only goes up. Property tests for scoring code. 2 or 3 Playwright tests and one extension test. OpenTelemetry with a local trace viewer. Nightly evals with model graders. An OWASP ASVS level 1 checklist. Feature flags in the settings.

CAUTION: GitHub secret scanning and push protection cost money on a private repository. Use gitleaks in the hooks and in CI until the budget allows the paid feature (S33).

19. Working with Claude Code

These rules keep the cost of each session low and keep AI-written code safe to merge.

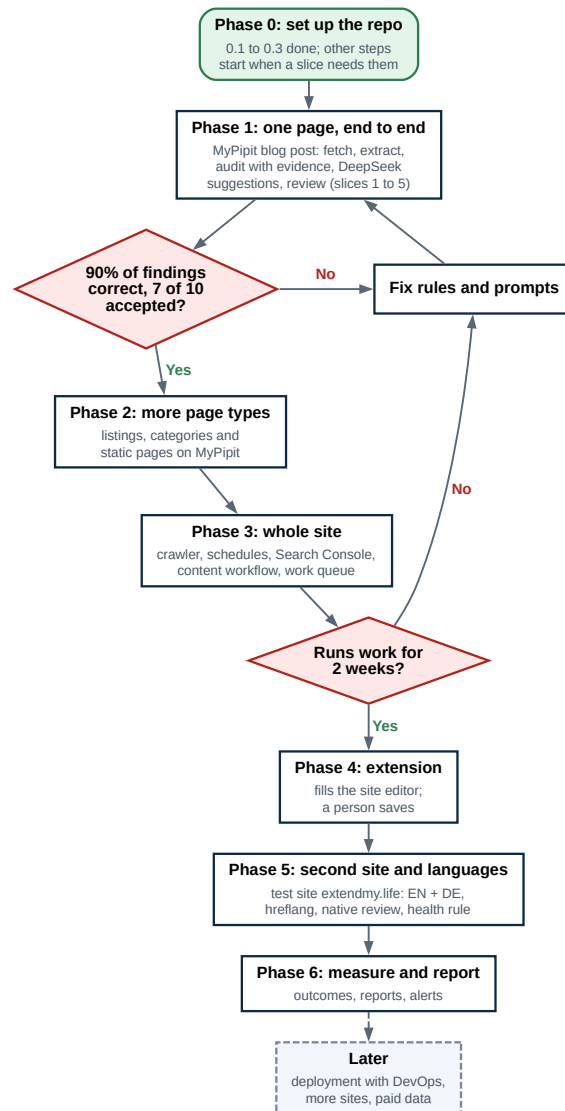
Topic	Rule
CLAUDE.md	CLAUDE.md and PLAN.md are at the repo root, not in the requirements folder. Keep it under 200 lines. Put in it the commands, the rules that differ from defaults and links to ADRs and PLAN.md. Do not copy long documents into it. Anthropic says that a long file makes Claude ignore instructions (S42).
Context and tokens	Load only the current step of PLAN.md. Do one task in each session and clear the context between tasks. Read only the research section that a task names. Use a subagent for wide searches. Do not load the PDF.
Plan first	Use plan mode for work that touches many files or is not clear. For a large feature, let Claude ask questions and write a short design document first.
Proof, not claims	Give Claude a check that it can run: tests, type check, build. Ask for the command and its output. A hook runs fast tests when Claude stops.
Tests first	Write a failing test, confirm that it fails, then write the code (S41).
Human review	A person reads every line before the merge. The pull request template asks if AI helped and who reviewed it.
Secrets	Deny rules stop Claude from reading .env files and keys. Keys go into the environment only. The sandbox is on.
Big files	Deny rules also stop reads of lock files, build output, node_modules, the generated client and PDFs. One lock file can cost thousands of tokens.
Quiet output	Commands and hooks print only failures, or the last 20 lines. A green run adds one line to the context.
Model and tools	Use a smaller model for routine edits and for search subagents. Keep the larger model for planning and hard bugs. Switch off MCP servers that the project does not use.
Untrusted text	Web pages and issue text can contain hidden instructions. Treat them as data. The AI that reads page text gets no tool that can write or send (S37).

Working rules for all engineers

Rule	Detail
Setup and checks	One setup command (<code>mise run setup</code>) and one check command (<code>mise run check</code>).
Pull requests	Keep a pull request small: aim for 400 changed lines or fewer. Squash merge. Write the title in Conventional Commits form, for example <code>feat(api): add sitemap reader</code> . Only the title is checked.
Approvals	0 approvals while one engineer works. 1 approval when a second engineer joins. Change the ruleset at that time.
Skipping a rule	Skip a check only with a comment that gives the reason, for example a type-ignore comment with its error code and reason.
Tasks	Tick a task in PLAN.md only when its "done when" is true. Log the time in Emitii on the matching card.

20. Order of work

The phases are the current direction and can change. Each phase is built in small slices (see How we work). In each slice, build and check the backend first, then its screen. Each phase ends with a check. If the check fails, fix the rules or prompts before the next phase. Estimates are for one engineer.



Phase	Done when	Estimate
0. Set up the repo	Steps 0.1 to 0.3 done. The other steps start when a slice needs them. The final check passes before a second person joins (step 0.12).	1 to 1.5 weeks in total
1. One page, end to end (MyPipit blog post)	Slices 1 to 5 are done. 90 percent or more of findings are correct. 7 of 10 suggestions are accepted with small edits.	2 to 3 weeks
2. More page types on MyPipit	Listings, blog categories and static pages have rule sets with sources. The page type also comes from the content.	1 to 2 weeks
3. Whole site	A full crawl is stored. Schedules run for 2 weeks without a manual start. Duplicate keywords are blocked. The work queue shows the top 10 with reasons.	2 to 3 weeks
4. Extension	A person applies and saves one approved suggestion. An item is Live only after the check confirms it.	1 to 2 weeks
5. Second site and languages (extendmy.life)	English and German pages match the 12 sitemaps. The health rule and native review are enforced in code.	2 to 3 weeks
6. Measure and report	Outcomes at 4, 8 and 12 weeks. A weekly report for each site owner.	1 to 2 weeks

21. Tasks, phases 1 to 3

Phase	Task	Done when
1	Slice 1: safe fetcher (SSRF guard, robots.txt, rate limit for each host; reused from SEOAdvisor), sitemap reader, page type and language for each URL. The sitemap list is now the page picker.	Done. The MyPipit URL list is the same as its sitemap URLs.
1	Slice 2a: database base, tables tenants and sites, <code>make seed</code>	Done.
1	Web app and API for slice 1: sites, sitemap list, request trace, page-type rules and URL tester, <code>make dev</code>	Done. The screens change in slice 2.
1	Slice 2b: save all sitemap pages	Removed (8 Oct 2026): only analysed pages are saved.
1	Slice 2: fetch and extract one page. Spike with 1 MyPipit blog post. <code>inventory.fetch_page</code> (the URL must be on a registered site), tables pages and page_snapshots, the extractor. Screen: analyse a page (paste a URL or pick one from the sitemap list).	The extracted fields of the fixture page are correct in tests. The screen shows them for a live MyPipit post.
1	Slice 3: audit one page. Rules for blog posts with sources, severity and evidence; tables audit_runs and findings; the keyword is part of the run. Screen: findings, filter by severity, check again.	Each finding has evidence and a source. Two runs of the same page can be compared.
1	Slice 4: AI suggestions. AI layer with DeepSeek (step 0.11); drafts: suggestions with [ADD: ...], Pydantic check, one retry; tables suggestions and llm_calls; checks for invented facts and copied text. Screen: accept, edit or reject; copy.	Every AI call is logged. No invented fact reaches review.
1	Slice 5: the loop on 5 MyPipit blog posts with the SEO team	90 percent or more of findings are correct. 7 of 10 suggestions are accepted with small edits.
2	Rules for listings (travel structured data), blog categories and static pages	Each type has its rule set with sources.
2	Page type from the content (title, structured data), not only the URL	The fixture pages get the correct type.
3	Inventory of all sitemap pages; crawler with robots.txt rules, rate limit for each host and snapshots	A full crawl is stored.
3	DBOS schedules (daily, weekly, monthly) and comparison with the last run	Runs work for 2 weeks without a manual start.
3	Search Console sync, keyword-to-page map with one primary keyword for each page and language, cannibalisation check	The platform blocks duplicates.
3	Content items, versions, states and roles; work queue: impact × confidence ÷ effort	The queue shows the top 10 items with reasons.

22. Tasks, phases 4 to 6, and later work

Phase	Task	Done when
4	Extension for the test site editor: show approved suggestions, fill on Accept, never save	A person applies and saves one approved suggestion.
4	Verify after publish: fetch the page again	An item becomes Live only after the check confirms it.
5	Second site and languages (test site extendmy.life): English and German pages, hreflang groups, page types	The pages match the 12 sitemaps.
5	Language rules (R2 §3) and health rules (R2 §1 and §2)	The known issues show as findings.
5	German text needs native review. The health rule is in code.	Nobody can skip either step.
6	Outcomes at 4, 8 and 12 weeks: per page first, then compared with unchanged pages of the same type (needs Phase 3)	The report shows outcomes for each change older than 4 weeks.
6	Weekly report and alerts (email or Emitii)	Each site owner gets one report each week.

Later (needs a decision first)	Note
Deployment	With DevOps: hosting, backups, login in front of the app.
Competitor analysis	Owner decision: later.
Paid keyword data	Only if free data is too thin.
Row-level security and outside accounts	When outside customers arrive.
GitHub Secret Protection and CodeQL	When the budget allows.
Sentry, SBOM, release automation	With deployment.
TypeScript 7, Python 3.15	After the tools support them.

22. Change log

When a strategy changes, strike the old task in PLAN.md, add a line to this change log, and update this PDF and the ADR.

Date	Change	Reason
5 Oct 2026	Plan created: phases 0 to 5, PostgreSQL, DBOS, read-only access with an extension	Requirements research (R1 to R3)
6 Oct 2026	Setup phase added: uv, pnpm, mise, strict checks, light process rules	Setup research (R4). Easy start for new engineers
6 Oct 2026	General tool. Small parts. Research, experiment, implement	Owner decision
6 Oct 2026	Modular monolith grouped by feature. Security rules in CLAUDE.md	Owner decision
6 Oct 2026	Review: repeated content removed. Session habits for people moved from CLAUDE.md to PLAN.md	Keep both files small
6 Oct 2026	CLAUDE.md kept as CLAUDE.template.md until it is copied. The PDF is rebuilt with every plan change. import-linter uses protected and forbidden contracts	Owner decision. Contract check
6 Oct 2026	PLAN.md and CLAUDE.md stay at the repo root. There is no CLAUDE.template.md	Owner decision
7 Oct 2026	Frontend: TanStack Router and Table, Tailwind CSS and shadcn/ui. React Hook Form and Zod in Phase 2. Recharts in Phase 5. Web folder layout	Owner decision
7 Oct 2026	.gitattributes and folder READMEs added to step 0.2. CONTRIBUTING.md and SECURITY.md moved from Should to step 0.2. A db:down task added	Owner decision
7 Oct 2026	Step 0.3: protected import contracts are added for each feature. pip-audit checks uv.lock. The pnpm checks join mise run check in step 0.6. The 7-day delay gives ruff 0.16.9 and mypy 2.3.1	One wildcard contract cannot limit a feature to its own internals. Owner decision
7 Oct 2026	Feature-first: Phase 1 in 5 slices. Steps 0.4 to 0.12 start when a slice needs them. Empty placeholder folders removed	Owner decision: see a real feature first
8 Oct 2026	Backend and frontend in parallel: every slice ends with its screen. API (step 0.5, without structlog) and web app (step 0.6) started early. make wraps the mise tasks. The fetcher records each request for the Activity screen. jsdom added for web tests. TypeScript 6.0.3 (openapi-typescript needs the TypeScript API)	Owner decision: see what each feature does
8 Oct 2026	Single page first: Phase 1 is one MyPipit blog post end to end (fetch, extract, audit with evidence, DeepSeek suggestions, review). The sitemap list is a page picker. The whole site moves to Phase 3, the extension to Phase 4, extendmy.life to Phase 5, reports to Phase 6. Backend first, then its screen, in each slice. Docs packages playwright and @hpcc-js/wasm-graphviz for mise run docs:pdf	Owner decision: do SEO on one page before the whole site

23. Open questions and risks

Item	Type	What to do
DeepSeek API key	Dependency	The owner gives the key before slice 4. It goes into the environment (Settings), never into code or git.
AI provider data terms	Open question	DeepSeek is chosen (8 Oct 2026). Read its data terms before slice 4 sends client page text to it.
extendmy.life editor and CMS	Open question	Find out which editor the authors use. The extension must support it.
Search Console for extendmy.life	Open question	Get read access to the domain property.
Health review rule	Owner decision	Decide before extendmy.life drafts start: expert reviewer or AI limited to non-medical text.
German native reviewer	Dependency	Name a person. Without one, German drafts stay in Draft.
German law for aesthetic clinics (HWG)	Legal	Ask a lawyer before drafts for aesthetic clinic pages.
Review cycle for health pages	Open question	Agree how often health pages need a review, for example every 6 months.
Code ownership	Open question	Find out if Gurzu or the client owns the code. This sets the NOTICE text.
One page is not the whole picture	Risk	Some checks need other pages (duplicate keywords, internal links to the page). The sitemap list helps a little. The full checks come in Phase 3.
Extension and editor changes	Risk	A weekly test checks that the editor fields still exist.
Small sites, few queries	Risk	Search Console hides rare queries. Add paid keyword data only if free data is too thin.
New tool versions	Risk	pnpm 12, Vitest 5 and SQLAlchemy 2.1 are new major versions. Lock files and Dependabot delays reduce the risk.
Expectations	Risk	No tool guarantees rank 1. Measure before and after each change.

24. Sources

Sources S1 to S24 were checked on 5 October 2026. Full notes with quotes are in the research folder (R1, R2, R3).

ID	Source	Trust
S1	Google, Search Quality Rater Guidelines, 11 Sep 2025. guidelines.raterhub.com/searchqualityevaluatorguidelines.pdf	Primary
S2	Google Search Central, "Creating helpful, reliable, people-first content" (updated 1 Oct 2026)	Primary
S3	Same guide: YMYL content "must be highly accurate and consistent with established expert consensus". Fabricated creator profiles are "a form of deception".	Primary
S4	Google Search Central, "Article structured data". schema.org MedicalClinic, MedicalWebPage, Physician	Primary
S5	Google Search Central, "Tell Google about localized versions of your page"	Primary
S6	Google Search Central, guidance on machine translation and scaled content (MyPipit evidence S12, S13)	Primary
S7	Google Search Central Blog, "Sitemaps ping endpoint is going away" (lastmod accuracy), 26 Jun 2023	Primary
S8	Google Search Central, "Managing crawling of faceted navigation URLs" (Dec 2024)	Primary
S9	Google Search Central, Google crawlers overview (2 MB HTML limit), JavaScript SEO basics	Primary
S10	IETF RFC 9309, Robots Exclusion Protocol	Primary
S11	Bing Webmaster Blog, "The importance of setting the lastmod tag" (Feb 2023)	Primary (Microsoft)
S12	web.dev, Core Web Vitals. Chrome for Developers, CrUX API	Primary
S13	Google Search Central Blog, Search Console weekly and monthly views (Dec 2025), custom annotations (Nov 2025)	Primary
S14	StatCounter, search engine market share Germany (Sep 2026)	Secondary
S15	IndexNow documentation. Bing Webmaster API access (learn.microsoft.com)	Primary
S16	GOV.UK, "Write change notes". GOV.WALES, "Peer reviewing content (2i)"	Primary
S17	Drupal Content Moderation overview. WordPress post status	Vendor or open source
S18	Brodersen et al. (2015), causal impact with Bayesian structural time-series models, Annals of Applied Statistics	Peer-reviewed
S19	MyPipit plan evidence: requirements/research/03-mypipit-evidence.md	Primary sources inside
S20	PostgreSQL 18 documentation. SQLite "Appropriate Uses" and "Write-Ahead Logging". pgvector README	Primary
S21	Martin Fowler, "Temporal Patterns" and "Event Sourcing". PostgreSQL trigger audit example	Secondary (expert)
S22	DBOS documentation (scheduled workflows, steps, testing). Procrastinate documentation	Vendor or open source
S23	AWS Prescriptive Guidance, multi-tenant SaaS partitioning models for PostgreSQL, and RLS	Vendor guidance
S24	Microsoft Azure Architecture Center, storage and data in multitenant solutions (2026-08-21)	Vendor guidance

25. Sources for the setup

Sources S25 to S45 were checked on 6 October 2026. Full notes, versions and unverified items are in research file R4.

ID	Source	Trust
S25	uv documentation (dependencies, locking, settings). PEP 621, PEP 735, PEP 751	Official, standard
S26	PEP 790, Python 3.15 release schedule. PyPI metadata for DBOS, FastAPI, Pydantic (checked 6 Oct 2026)	Standard, official
S27	Ruff documentation. "Mypy 2.0 Released" (May 2026). Pyrefly 1.0 blog. Astral ty blog	Official
S28	pytest documentation, configuration (pytest 9 TOML config and strict mode). Testcontainers for Python guide	Official, vendor
S29	"Vite 8.0 is out!" (March 2026). "Announcing TypeScript 7.0" (July 2026). create-vite react-ts template. ESLint v10 release	Official
S30	WXT documentation. Playwright "Chrome extensions" guide	Official
S31	mise documentation. Thoughtworks Technology Radar Vol 34 (April 2026)	Official, vendor opinion
S32	Docker library documentation for PostgreSQL 18 (new data directory). PostgreSQL 18 release notes	Official
S33	GitHub Docs: rulesets, secure use of Actions, licensing. GitHub Changelog: SHA-pinning policy (Aug 2025), Dependabot uv support (Mar 2025) and cooldown (Jul 2026), Secret Protection and Code Security (Mar 2025)	Vendor
S34	Conventional Commits 1.0.0. DORA, trunk-based development and delivery metrics (updated Jan 2026)	Standard, official
S35	MADR 4.0.0 (adr.github.io). C4 model (c4model.com). Diátaxis (diataxis.fr)	Official
S36	Google Engineering Practices: "Small CLs" and "The Standard of Code Review"	Official (Google)
S37	OWASP ASVS 5.0. NIST SP 800-218 (SSDF 1.1). OWASP SSRF Prevention Cheat Sheet. OWASP Top 10 for LLM Applications 2025	Standard
S38	structlog "Logging Best Practices". OpenTelemetry Python documentation	Official
S39	"SQLAlchemy 2.1.0 Released" (Sep 2026). Alembic "The Importance of Naming Constraints". RFC 9562 (UUIDs)	Official, standard
S40	RFC 9457 Problem Details for HTTP APIs. Google AIP-158 (pagination), AIP-185 (versioning). FastAPI pull request 15951 (closed, not merged)	Standard, official
S41	Anthropic, "Building effective agents" (Dec 2024) and "Demystifying evals for AI agents" (Jan 2026). Simon Willison, "Agentic Engineering Patterns" (Feb 2026)	Vendor, secondary
S43	Kirsten Westeinde, Shopify Engineering, "Deconstructing the Monolith: Designing Software that Maximizes Developer Productivity" (21 Feb 2019). Quote: "re-organize it by real-world concepts (like orders, shipping, inventory, and billing)"	Secondary (company engineering blog)
S44	Jimmy Bogard, "Vertical Slice Architecture" (19 Apr 2018). Quote: "Minimize coupling between slices, and maximize coupling in a slice."	Secondary (expert)
S45	import-linter 2.15 (4 Sep 2026, BSD licence) on PyPI and its documentation: contract types forbidden, protected, layers, independence, acyclic siblings	Official
S42	Claude Code documentation: "Best practices", "How Claude remembers your project" (target under 200 lines), hooks, skills, subagents, security	Vendor